

Architecture Logicielle – Detecteur Pro

Une architecture **frontend-first**, modulaire et orientée composants, conçue pour simuler en temps réel un système de sécurité intelligent. Ce document présente la structure technique, les flux de données, les modules principaux et les choix d'implémentation qui font de Detecteur Pro une application robuste, évolutive et maintenable.

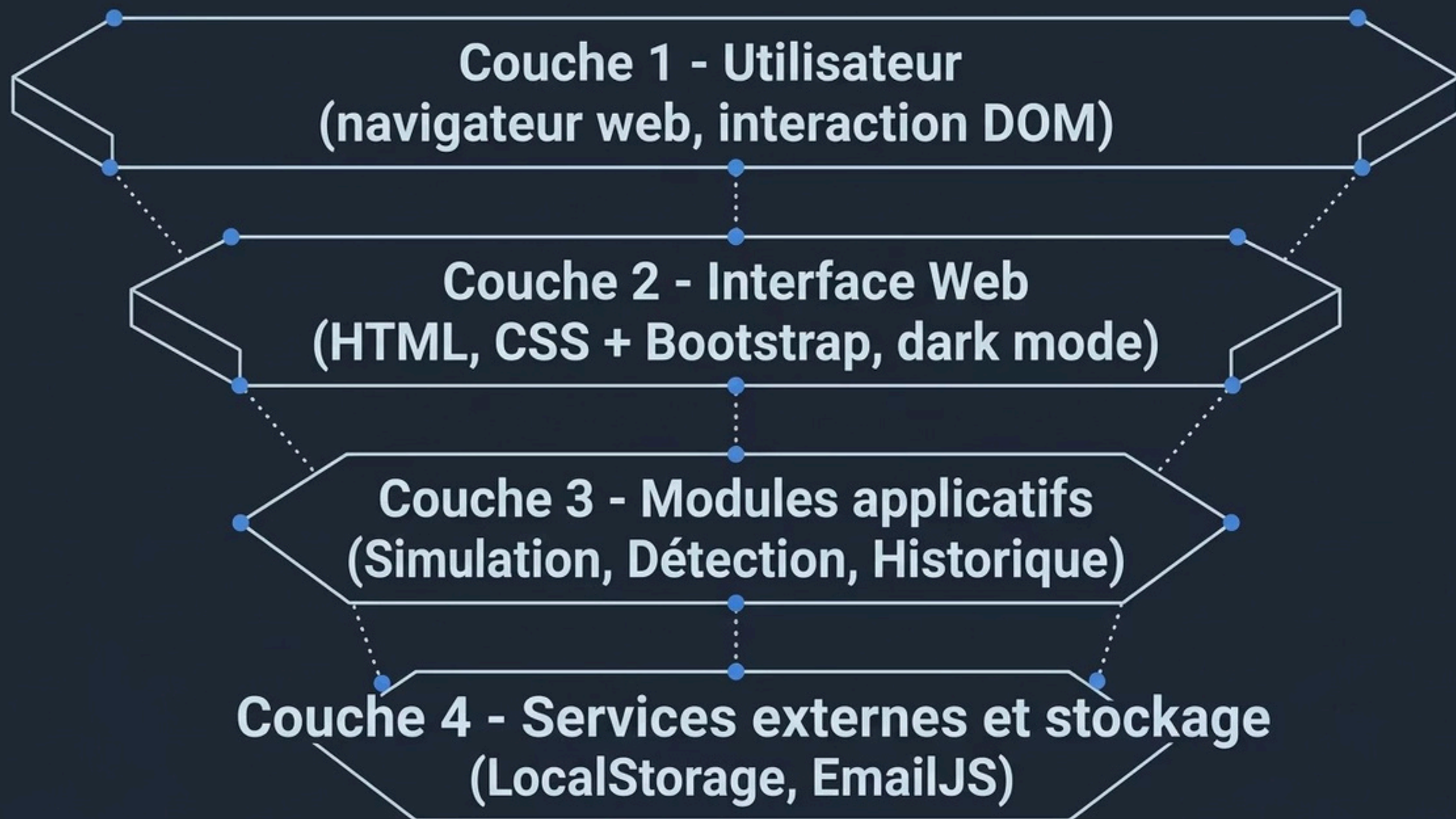
 FRONTEND-FIRST

 MODULAIRE

 EVENT-DRIVEN

Vue d'ensemble de l'architecture

L'architecture de DetecteurPro repose sur un modèle **frontend-first** : aucun backend serveur n'est requis. L'ensemble de la logique applicative, du stockage et de la communication est géré côté client, à travers des modules JavaScript indépendants et communicants.



Chaque couche a une responsabilité claire, garantissant la **séparation des préoccupations** et facilitant la maintenance à long terme.

Stack Technique Frontend

DetecteurPro s'appuie sur un ensemble de technologies web standards, soigneusement choisies pour leur légèreté, leur compatibilité et leur capacité à produire une interface moderne et réactive.



HTML5

Structure sémantique de l'application. Balisage accessible, organisation logique des sections et des composants UI.



CSS3 + Bootstrap

Design responsive, thème dark mode, animations fluides et grille adaptative pour tous les écrans.



JavaScript ES6+

Logique applicative, gestion des événements, manipulation du DOM et orchestration inter-modules.



LocalStorage

Persistance des données côté client sans base de données. Sauvegarde de l'historique des événements de détection.



EmailJS

Envoi d'emails directement depuis le frontend, sans backend ni serveur SMTP dédié.



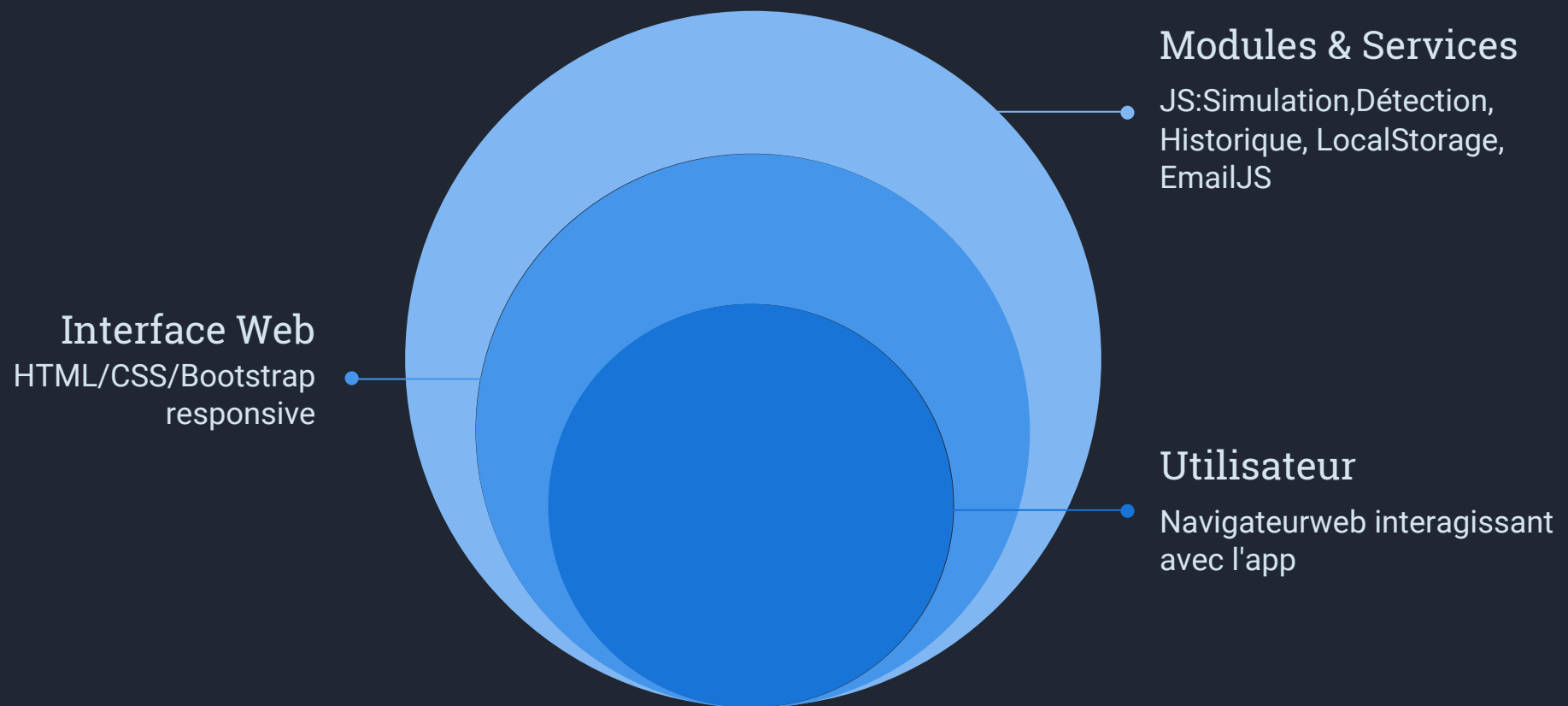
Modules Principaux

L'application est découpée en **quatre modules fonctionnels indépendants**. Chacun encapsule sa propre logique, ses propres états et ses interactions avec le DOM, tout en pouvant communiquer avec les autres via des événements JavaScript natifs.

- | | |
|---|--|
| 1 | <h2>Simulation</h2> <p>Rendu animé de la maison, du détecteur et de l'intrus. Gestion de l'état actif/inactif du système en temps réel.</p> |
| 2 | <h2>Détection</h2> <p>Analyse des zones surveillées, déclenchement des alertes visuelles et sonores dès qu'une intrusion est identifiée.</p> |
| 3 | <h2>Historique</h2> <p>Enregistrement horodaté de chaque événement de sécurité. Lecture et écriture dans le LocalStorage du navigateur.</p> |
| 4 | <h2>Contact</h2> <p>Formulaire de contact intégré. Envoi automatique via EmailJS avec confirmation à l'utilisateur.</p> |

Diagramme d'Architecture Système

Ce diagramme illustre les relations entre l'utilisateur, l'interface web, les modules applicatifs et les services de persistance et de communication. Les modules sont **faiblement couplés** et communiquent via des événements JavaScript.



La communication entre modules est entièrement gérée par le **système d'événements JavaScript**, sans dépendance directe entre eux — ce qui facilite les tests unitaires et les évolutions futures.

Flux de Données – Détection d'Intrusion

Le flux principal de l'application suit un pipeline linéaire déclenché par l'activation du système. Chaque étape est traitée de manière synchrone ou via des callbacks d'événements, garantissant une réactivité optimale de l'interface.



Réactivité temps réel

Chaque événement DOM déclenche une chaîne de callbacks immédiate, sans latence réseau grâce à l'architecture 100% frontend.

Gestion d'état

L'état global du système (actif, alerte, historique) est maintenu en mémoire JavaScript et synchronisé avec le LocalStorage à chaque changement significatif.

Flux de Communication – Module Contact

Le module Contact implémente un flux de communication asynchrone permettant à l'utilisateur d'envoyer des messages sans qu'aucun serveur backend ne soit nécessaire. EmailJS agit comme passerelle entre le frontend et les serveurs SMTP.



Saisie Utilisateur

L'utilisateur remplit le formulaire de contact : nom, email, message.



Validation JS

Le module JavaScript valide les champs côté client avant tout envoi.



Envoi EmailJS

Appel API EmailJS depuis le frontend. Aucun backend requis.



Confirmation

Retour visuel immédiat à l'utilisateur : succès ou message d'erreur.



EmailJS permet d'envoyer jusqu'à 200 emails/mois en version gratuite, suffisant pour un prototype ou une application de démonstration.

Interface Utilisateur – Design & Expérience

L'interface de DetecteurPro est pensée pour être **intuitive, moderne et accessible**. Le thème dark mode réduit la fatigue visuelle lors de la surveillance prolongée, tandis que le design responsive garantit une expérience cohérente sur tous les appareils.



Responsive & Dark Mode

Bootstrap assure l'adaptation automatique aux tailles d'écran. Le thème sombre est conçu pour les environnements de surveillance.



Cartes Interactives

Chaque module est présenté sous forme de carte interactive avec animations CSS3 au survol et transitions fluides.



Animations Temps Réel

La simulation visuelle de l'intrus et des zones de détection utilise des animations JavaScript et CSS pour un rendu immersif.

Stockage & Persistance – LocalStorage

En l'absence de backend, le **LocalStorage** du navigateur constitue la couche de persistance de `DetecteurPro`. Il stocke l'historique des événements de détection sous format JSON, permettant leur consultation même après rechargement de la page.

Structure des données stockées

```
{
  "events": [
    {
      "id": "evt_001",
      "type": "intrusion",
      "zone": "Zone Nord",
      "timestamp": "2024-01-15T14:32:00Z",
      "status": "alerte"
    },
    {
      "id": "evt_002",
      "type": "activation",
      "timestamp": "2024-01-15T14:30:00Z",
      "status": "système actif"
    }
  ]
}
```

Caractéristiques

- Capacité : jusqu'à ~5 MB par domaine
- Persistance entre sessions navigateur
- Accès synchrone et instantané
- Sérialisation JSON native
- Aucune dépendance externe
- Suppression programmée possible

Évolutivité & Perspectives

L'architecture modulaire et event-driven de DetecteurPro est conçue pour **évoluer sans refonte majeure**. L'ajout de nouvelles fonctionnalités se fait en greffant de nouveaux modules sur le système d'événements existant.



Fondation actuelle

Architecture modulaire frontend-first avec LocalStorage et EmailJS. Base solide et fonctionnelle.



Intégration Backend

Ajout d'une API REST (Node.js / Firebase) pour synchronisation cloud et multi-utilisateurs.



Intelligence Artificielle

Module IA pour la détection prédictive d'intrusions via analyse comportementale et machine learning.



Notifications Temps Réel

WebSockets ou Server-Sent Events pour alertes push instantanées sur mobile et desktop.



La séparation stricte des responsabilités entre modules garantit que chaque évolution peut être développée et testée indépendamment, sans impacter l'application existante.